TempoSync | Minimum Viable Product

Implementation-ready product requirement · Rev. 2 (October 15, 2026)

Product definition

TempoSync is a responsive workout music web app that asks what the user is doing, learns stated genre preferences by activity, and selects/transitions between authorized tracks as workout states change.

Primary audience: recreational runners and gym-goers. Primary promise: music transitions respond to workout intent and events, not merely playlist order, and land on the song's drop when the user needs a push. MVP does not promise measurable performance improvement.

Signal honesty: in the MVP, running pace comes from a simulator or, optionally, live phone GPS (P1); strength events come from one-tap controls. TempoSync does not detect reps or exertion automatically, and the pitch should say so plainly.

MVP goals and non-goals

Goals: complete onboarding in under two minutes; start a workout; hear a track; manually control sets/rest or simulate running pace (optionally use live GPS pace outdoors); receive relevant, explainable song changes, with boost switches landing on the track's drop; save preferences locally.

Non-goals: live Spotify catalog integration, Apple Watch/Fitbit data, automated rep detection, camera/microphone analysis, music time-stretching or audio editing (seeking to a timestamp is allowed), generative music, native mobile apps, real ML model training, user accounts, social features.

User stories

- As a runner, I set target pace and preferred running genres so TempoSync can select music that fits my intended session.
- As a runner outdoors, I can switch the pace source to live GPS so I don't have to touch my phone mid-run.
- As a lifter, I enter exercises and sets, then mark a set complete so rest music plays automatically.
- As a user, I can request 'boost', 'calm', 'skip', or 'keep this song' and override automated suggestions.
- As a user, when I ask for a boost or my pace drifts, the next song enters at (or just before) its drop so the change is felt immediately.
- As a user, I see why a track was chosen and can pause all automatic switching.
- As a demo operator, I can simulate pace and exertion without a wearable.

Screens and flows

1. Welcome and activity selection

Select Running or Strength. Provide concise statement that the app uses manually entered or simulated signals.

2. Guided setup

Activity-specific genre, 3–4 example songs or artists as text, intensity preference, goal; for running target min/mile, for strength exercise list, sets, rest duration. Seed artists are matched against catalog artist names and earn a preference bonus; seeds not in the demo catalog are listed as "not in demo catalog" rather than silently ignored.

3. Workout dashboard

Large music player, current track, genre/BPM/energy tags, drop marker on the progress bar when tagged, workout status, pace source (Simulated/GPS), elapsed time, decision explanation, auto-DJ toggle.

4. Activity controls

Running: pace source toggle (Simulated / GPS), pace slider (simulated mode), and high-exertion toggle. Strength: Start set, Set complete, Start next set/exercise. Global: Boost, Calm, Skip, Pause.

5. Session summary

Events, tracks played, transitions, preference feedback; no fabricated calories, health insights, or performance gains.

Functional requirements

FR-01: Profile stores per-activity genre preferences, seed artist/track text, and intensity in localStorage.

FR-02: Workout session initializes a WorkoutPlan (activity, goal, targetPace or exercises), the initial WorkoutState, and the demo audio catalog.

FR-03: Decision engine evaluates events and returns a Decision: action (keep/switch), next track, transition mode, start offset, and reason.

FR-04: Browser player supports pause/play, next, volume, mid-track cut, 2–4 second crossfade, and starting a track at a given offset (startAtSec).

FR-05: Running pace input (simulated or GPS) triggers a drift event only when pace is at least 0:30 min/mile off target for at least 15 seconds. Slower than target may raise desired energy by one level; faster than target holds energy (never escalates); high exertion lowers it. Brief jitter must not cause track thrashing.

FR-06: Strength workflow accepts exercises, set completion, rest, and next exercise.

FR-07: User can disable auto-DJ and override any track choice.

FR-08: UI logs transitions and reasons, including when no suitable track is available.

FR-09: Accessible controls with keyboard interaction, clear status labels, and mobile-responsive layout.

FR-10 (P1): Live GPS pace via the browser Geolocation API (watchPosition), smoothed over a rolling 30-second window and emitted as the same pace events the simulator produces. The simulator remains the default for indoor demos; GPS requires HTTPS and location permission, and falls back to the simulator on denial or poor signal.

FR-11 (P1): Drop-sync. Tracks may carry a hand-tagged dropAtSec. For escalation and Boost switches, the engine sets startAtSec = max(0, dropAtSec − crossfadeSec) so the drop lands as the crossfade completes. Rest/calm switches and untagged tracks start at 0.

Recommended technology stack

Frontend: React 18+ / TypeScript / Vite; Tailwind CSS for responsive UI; Zustand or React context for session state; Web Audio API + HTMLAudioElement for crossfades.

Backend: FastAPI (Python) only if needed for recommendation and future integrations; Pydantic request/response schemas; pytest for backend rules. Start frontend-only if time constrained, but keep engine isolated behind a typed interface.

Data: bundled JSON track catalog (target 12–16 tracks, minimum 8; expand toward ~50 after the event) using Creative Commons or royalty-free audio with the license recorded per track; localStorage for preferences and session settings. Deployment: Vercel/Netlify frontend (HTTPS is required for GPS); Render/Fly.io backend if used. GitHub Actions for tests optional.

AI tools: Cursor for iterative implementation, Codex for bounded components/tests, Claude for specification review. Keep one source-of-truth issue list and avoid concurrent edits to the same files.

Minimal API contract

POST /api/recommend: input {profile,plan,workoutState,event,currentTrackId,recentTrackIds}; output {action:'keep'|'switch',trackId?,transition:'none'|'cut'|'crossfade',startAtSec?,reason}. These shapes are shared with the POC document and are the single source of truth.

GET /api/tracks: returns eligible local catalog metadata (never copyrighted streaming URLs). GET /health: status. If frontend-only, implement equivalent typed functions and preserve the request/response shapes.

Recommendation algorithm

1. Compute desired energy (1–5) from activity + state + user intensity. 2. Apply goal signal: pace at least 0:30 min/mile slower than target for 15 seconds raises desired energy by one; faster than target holds it; rest or high exertion lowers it. 3. Filter catalog by authorized audio availability and activity preferences; exclude the current track. 4. Score candidates: energy fit (40%), genre and seed-artist match (30%), BPM proximity (20%), novelty (10%). Weights are tunable product heuristics, not research-validated. 5. Enforce a 30-second cooldown except for explicit user commands and set/rest state transitions. 6. Choose the start offset: drop-sync for escalations and Boost, otherwise 0. 7. Return an explainable action, transition mode, and start offset; if no eligible track exists, keep the current track and say why.

Use deterministic fixture data and unit tests. Do not claim that higher BPM always improves pace or that a given track is guaranteed to motivate a user.

Core entities

Profile(id, preferencesByActivity, seedArtists, seedTracks, intensity); Track(id, title, artist, genre, bpm, energy 1–5, durationSec, dropAtSec?, audioUrl, licenseNote); WorkoutPlan(activity, goal, targetPace?, exercises?, restSec?); WorkoutState(status, paceSource 'simulated'|'gps', exerciseIndex?, setIndex?, currentPace?, exertionFlag?, driftSince?, lastSwitchAt); WorkoutEvent(type, at, payload); Decision(action, trackId?, transition, startAtSec?, reason, confidence?).

Definition of done

- Fresh user can complete setup and play demo music in Chrome/Safari mobile or desktop.

- Running drift scenario (at least 0:30 min/mile slower for 15 seconds) produces exactly one appropriate, logged transition; running faster than target does not escalate.
- Strength set-complete crossfades into rest and next-set returns to higher-energy music.
- A Boost switch to a drop-tagged track lands its drop within about one second of the crossfade completing.
- Catalog of 12–16 authorized demo tracks (minimum 8) spanning two genres and three energy bands, with at least four high-energy tracks tagged with dropAtSec.
- No infinite skips, no audio overlap after crossfade, no auto-play without interaction.
- If built, GPS mode produces pace readings outdoors and falls back to the simulator on permission denial or poor signal.
- At least 10 unit tests for recommendation rules and 3 end-to-end/manual demo scenarios.
- README explains setup, architecture, track licensing, demo mode, known limitations, and which parts were built before vs. during HackOHI/O.

Backlog priority

P0 / Must: activity setup, local catalog, music player, workout event controls (simulated), decision engine, basic cut transitions, explanation UI.

P1 / Should: crossfade + drop-sync (the signature demo moment), live GPS pace, activity-specific preferences with seed-artist matching, pace drift hysteresis, rest timer, event log, tests.

P2 / Could: natural-language setup (an LLM parses "hip-hop for lifting, lo-fi for runs" into Profile JSON, with the form as fallback and the API key kept server-side), accelerometer step cadence via DeviceMotion, session history, richer song feedback, optional FastAPI service.

Post-MVP: expand the catalog toward ~50 tagged tracks, wearable heart rate/motion, automatic set detection, personalized preference ranking based on opt-in feedback, third-party streaming after legal/technical review.

Delivery sequence and ownership

Milestone 1: scaffold + catalog (with drop tags) + reliable playback. Milestone 2: state machine + recommendation rules + tests. Milestone 3: running and strength screens. Milestone 4: crossfade + drop-sync + accessibility. Milestone 5: GPS pace (P1), demo rehearsal and deployment.

Event context: HackOHI/O 12 runs October 24–25, 2026 (24 hours). The event allows pre-built components but asks teams to be honest and transparent with judges about what was accomplished during the event. Keep commit history intact, list pre-event work in the README and pitch, and consider saving one visible, demo-able piece (e.g., GPS pace or natural-language setup) for the event itself.

Solo owner: focus on P0, then crossfade + drop-sync, before any chatbot/LLM. Two-person split: Person A frontend/audio/UX; Person B engine/state/API/tests. Pair on interface definitions, integration, and demo. Avoid dependency on third-party streaming approvals.

Instructions to AI implementation agent

Build the TempoSync MVP exactly as specified. First propose a repository structure and milestone plan. Then implement a running, testable Vite/React/TypeScript application using a bundled JSON track catalog and user-provided or openly licensed audio. Use a deterministic recommendation service, typed workout events, running simulation (with GPS pace behind the same pace-event interface), strength set/rest controls, cut and crossfade transitions with drop-sync start offsets, manual override, and clear explanations. Keep secrets out of the repository. Include fixtures, tests,

README, and commands to run locally. Flag any missing audio assets or missing dropAtSec tags rather than inventing URLs or timestamps or using unlicensed music. Do not implement Spotify integration unless explicitly requested later.